

```
// Experiment 1
```

```
public class Experiment1 {
    static void showHierarchy(Class<?> cls) {
        System.out.println(cls.getSimpleName());
        Class<?> parent = cls.getSuperclass();
        while (parent != null) {
            System.out.print(" -> " + parent.getSimpleName());
            parent = parent.getSuperclass();
        }
        System.out.println();
    }

    static void demonstrate(String name, Runnable action) {
        try {
            action.run();
        } catch (ArithmeticException e) {
            System.out.println(name + ": ArithmeticException - " + e.getMessage());
        } catch (NullPointerException e) {
            System.out.println(name + ": NullPointerException");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(name + ": ArrayIndexOutOfBoundsException");
        } catch (NumberFormatException e) {
            System.out.println(name + ": NumberFormatException - " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("Exception hierarchy:");
        showHierarchy(ArithmeticException.class);
        showHierarchy(NullPointerException.class);
        showHierarchy(ArrayIndexOutOfBoundsException.class);
        showHierarchy(NumberFormatException.class);

        demonstrate("Division", () -> System.out.println(10 / 0));
        demonstrate("Null access", () -> {
            String s = null;
            System.out.println(s.length());
        });
        demonstrate("Array access", () -> {
            int[] a = {1, 2, 3};
            System.out.println(a[5]);
        });
        demonstrate("Number conversion", () -> System.out.println(Integer.parseInt("abc")));
    }
}
```

// Experiment 2

```
public class Experiment2 {
    public static void main(String[] args) {
        try {
            System.out.println("Outer try started.");
            try {
                int[] values = {10, 20};
                System.out.println(values[5]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: array index is invalid.");
            }

            try {
                int result = 10 / 0;
                System.out.println(result);
            } catch (ArithmeticException e) {
                System.out.println("Inner catch: cannot divide by zero.");
            }

            int[] values = {1, 2, 3};
            System.out.println(values[10]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Outer catch: array index is invalid.");
        } catch (Exception e) {
            System.out.println("Outer catch: " + e.getClass().getSimpleName());
        } finally {
            System.out.println("Finally: outer block completed.");
        }
    }
}
```

```

// Experiment 3

import java.util.Scanner;

public class Experiment3 {
    static class InvalidOperatorException extends Exception {
        InvalidOperatorException(String message) { super(message); }
    }

    static double calculate(double a, double b, String op) throws InvalidOperatorException {
        switch (op) {
            case "+": return a + b;
            case "-": return a - b;
            case "*": return a * b;
            case "/":
                if (b == 0) throw new ArithmeticException("Division by zero is not allowed.");
                return a / b;
            default: throw new InvalidOperatorException("Invalid operator: " + op);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            double a = Double.parseDouble(sc.nextLine());
            System.out.print("Enter second number: ");
            double b = Double.parseDouble(sc.nextLine());
            System.out.print("Enter operator (+, -, *, /): ");
            String op = sc.nextLine().trim();
            System.out.println("Result = " + calculate(a, b, op));
        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid numeric input.");
        } catch (ArithmeticException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (InvalidOperatorException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}

```

```

// Experiment 4

import java.util.Scanner;

public class Experiment4 {
    static class InvalidMarksException extends Exception {
        InvalidMarksException(String message) { super(message); }
    }

    static void validateMarks(int marks) throws InvalidMarksException {
        if (marks < 0 || marks > 100) {
            throw new InvalidMarksException("Marks must be between 0 and 100. Given: " + marks);
        }
    }

    static char getGrade(double percentage) {
        if (percentage >= 90) return 'A';
        if (percentage >= 80) return 'B';
        if (percentage >= 70) return 'C';
        if (percentage >= 60) return 'D';
        if (percentage >= 50) return 'E';
        return 'F';
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter number of subjects: ");
            int n = Integer.parseInt(sc.nextLine());
            if (n <= 0) throw new IllegalArgumentException("Number of subjects must be positive.");

            int total = 0;
            for (int i = 1; i <= n; i++) {
                System.out.print("Enter marks for subject " + i + ": ");
                int marks = Integer.parseInt(sc.nextLine());
                validateMarks(marks);
                total += marks;
            }

            double percentage = (double) total / n;
            System.out.println("Total = " + total);
            System.out.printf("Percentage = %.2f%%\n", percentage);
            System.out.println("Grade = " + getGrade(percentage));
        } catch (InvalidMarksException e) {
            System.out.println("Error: " + e.getMessage());
            System.out.println("Result is not calculated because input is invalid.");
        } catch (NumberFormatException e) {
            System.out.println("Error: Please enter integer marks only.");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}

```

```
// Experiment 5

import java.util.HashMap;
import java.util.Map;

public class Experiment5 {
    static class InsufficientBalanceException extends Exception {
        InsufficientBalanceException(String message) { super(message); }
    }
    static class InvalidAmountException extends Exception {
        InvalidAmountException(String message) { super(message); }
    }
    static class AccountNotFoundException extends Exception {
        AccountNotFoundException(String message) { super(message); }
    }

    static class Bank {
        private final Map<String, Double> accounts = new HashMap<>();

        Bank() {
            accounts.put("1001", 5000.0);
            accounts.put("1002", 2500.0);
        }

        private double getBalance(String account) throws AccountNotFoundException {
            if (!accounts.containsKey(account)) {
                throw new AccountNotFoundException("Account " + account + " not found.");
            }
            return accounts.get(account);
        }

        void deposit(String account, double amount)
            throws AccountNotFoundException, InvalidAmountException {
            getBalance(account); // demonstrates propagation from one method to another
            if (amount <= 0) throw new InvalidAmountException("Deposit must be greater than zero.");
            accounts.put(account, accounts.get(account) + amount);
        }

        void withdraw(String account, double amount)
            throws AccountNotFoundException, InvalidAmountException, InsufficientBalanceException {
            double balance = getBalance(account); // exception propagates to caller
            if (amount <= 0) throw new InvalidAmountException("Withdrawal must be greater than zero.");
            if (amount > balance) {
                throw new InsufficientBalanceException("Insufficient balance. Current balance: " + balance);
            }
            accounts.put(account, balance - amount);
        }

        double balance(String account) throws AccountNotFoundException {
            return getBalance(account);
        }
    }

    public static void main(String[] args) {
        Bank bank = new Bank();
        try {
            bank.deposit("1001", 1000);
            System.out.println("Deposit successful. Balance = " + bank.balance("1001"));

            bank.withdraw("1001", 2500);
            System.out.println("Withdrawal successful. Balance = " + bank.balance("1001"));

            bank.withdraw("1001", 10000);
        } catch (InsufficientBalanceException e) {
            System.out.println("Transaction error: " + e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println("Transaction error: " + e.getMessage());
        } catch (AccountNotFoundException e) {
            System.out.println("Transaction error: " + e.getMessage());
        }

        try {
            bank.deposit("9999", 500);
        } catch (AccountNotFoundException e) {
            System.out.println("Account error: " + e.getMessage());
        } catch (InvalidAmountException e) {
            System.out.println("Amount error: " + e.getMessage());
        }
    }
}
```

// Experiment 6

import java.util.Scanner;

```
public class Experiment6 {
    static class InvalidUsernameException extends Exception {
        InvalidUsernameException(String message) { super(message); }
    }
    static class InvalidPasswordException extends Exception {
        InvalidPasswordException(String message) { super(message); }
    }
    static class AccountLockedException extends Exception {
        AccountLockedException(String message) { super(message); }
    }

    static class LoginSystem {
        private final String correctUsername = "admin";
        private final String correctPassword = "java123";
        private int failedAttempts = 0;
        private final int maxAttempts = 3;
        private boolean locked = false;

        void login(String username, String password)
            throws InvalidUsernameException, InvalidPasswordException, AccountLockedException {
            if (locked) throw new AccountLockedException("Account is locked. Too many failed attempts.");
            if (!correctUsername.equals(username)) {
                registerFailure();
                throw new InvalidUsernameException("Invalid username.");
            }
            if (!correctPassword.equals(password)) {
                registerFailure();
                throw new InvalidPasswordException("Invalid password.");
            }
            failedAttempts = 0;
            System.out.println("Login successful. Welcome, " + username + "!");
        }

        private void registerFailure() throws AccountLockedException {
            failedAttempts++;
            if (failedAttempts >= maxAttempts) {
                locked = true;
                throw new AccountLockedException("Maximum failed attempts reached. Account locked.");
            }
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    LoginSystem system = new LoginSystem();
    try {
        for (int attempt = 1; attempt <= 3; attempt++) {
            try {
                System.out.print("Username: ");
                String username = sc.nextLine();
                System.out.print("Password: ");
                String password = sc.nextLine();
                system.login(username, password);
                break;
            } catch (InvalidUsernameException e) {
                System.out.println("Login failed: " + e.getMessage());
            } catch (InvalidPasswordException e) {
                System.out.println("Login failed: " + e.getMessage());
            } catch (AccountLockedException e) {
                System.out.println("Security error: " + e.getMessage());
                break;
            }
        }
    } finally {
        sc.close();
        System.out.println("Login process finished.");
    }
}
```

// Experiment 7

```
import java.util.ArrayList;
import java.util.List;

public class Experiment7 {
    static class ApplicationException extends Exception {
        ApplicationException(String message) { super(message); }
    }
    static class ProductException extends ApplicationException {
        ProductException(String message) { super(message); }
    }
    static class ProductNotFoundException extends ProductException {
        ProductNotFoundException(String message) { super(message); }
    }
    static class OutOfStockException extends ProductException {
        OutOfStockException(String message) { super(message); }
    }
    static class PaymentException extends ApplicationException {
        PaymentException(String message) { super(message); }
    }
    static class InvalidPaymentException extends PaymentException {
        InvalidPaymentException(String message) { super(message); }
    }
    static class InsufficientFundsException extends PaymentException {
        InsufficientFundsException(String message) { super(message); }
    }
    static class OrderException extends ApplicationException {
        OrderException(String message) { super(message); }
    }
    static class EmptyCartException extends OrderException {
        EmptyCartException(String message) { super(message); }
    }
}

static class Product {
    String id, name;
    double price;
    int stock;
    Product(String id, String name, double price, int stock) {
        this.id = id; this.name = name; this.price = price; this.stock = stock;
    }
}

static class Cart {
    private final List<Product> items = new ArrayList<>();

    void addProduct(Product p) throws OutOfStockException {
        if (p.stock <= 0) throw new OutOfStockException(p.name + " is out of stock.");
        items.add(p);
        p.stock--;
        System.out.println(p.name + " added to cart.");
    }

    void removeProduct(String id) throws ProductNotFoundException {
        for (int i = 0; i < items.size(); i++) {
            if (items.get(i).id.equals(id)) {
                Product p = items.remove(i);
                p.stock++;
                System.out.println(p.name + " removed from cart.");
                return;
            }
        }
        throw new ProductNotFoundException("Product " + id + " is not in the cart.");
    }

    double total() throws EmptyCartException {
        if (items.isEmpty()) throw new EmptyCartException("Cart is empty.");
        double sum = 0;
        for (Product p : items) sum += p.price;
        return sum;
    }
}

static Product search(List<Product> products, String id) throws ProductNotFoundException {
    for (Product p : products) if (p.id.equals(id)) return p;
    throw new ProductNotFoundException("Product " + id + " not found.");
}

static void makePayment(double amount, double funds, String method)
    throws InvalidPaymentException, InsufficientFundsException {
    if (!(method.equalsIgnoreCase("card") || method.equalsIgnoreCase("upi"))))
        throw new InvalidPaymentException("Supported payment methods are Card and UPI.");
    if (amount <= 0) throw new InvalidPaymentException("Payment amount must be positive.");
    if (funds < amount) throw new InsufficientFundsException("Insufficient funds for payment.");
    System.out.println("Payment successful using " + method.toUpperCase() + ".");
}

public static void main(String[] args) {
    List<Product> products = new ArrayList<>();
    products.add(new Product("P101", "Keyboard", 1200, 2));
}
```

```

products.add(new Product("P102", "Mouse", 700, 0));
products.add(new Product("P103", "Headphones", 1800, 3));

Cart cart = new Cart();
try {
    Product p1 = search(products, "P101");
    cart.addProduct(p1);
    Product p2 = search(products, "P102");
    cart.addProduct(p2); // throws OutOfStockException
} catch (ProductNotFoundException e) {
    System.out.println("Product error: " + e.getMessage());
} catch (OutOfStockException e) {
    System.out.println("Stock error: " + e.getMessage());
}

try {
    double total = cart.total();
    System.out.println("Cart total = " + total);
    makePayment(total, 2000, "card");
    System.out.println("Order placed successfully.");
} catch (EmptyCartException e) {
    System.out.println("Order error: " + e.getMessage());
} catch (InvalidPaymentException e) {
    System.out.println("Payment error: " + e.getMessage());
} catch (InsufficientFundsException e) {
    System.out.println("Payment error: " + e.getMessage());
}

try {
    search(products, "P999");
} catch (ProductNotFoundException e) {
    System.out.println("Search error: " + e.getMessage());
}
}
}

```